TaskWA

Installation Guide

From an empty computer to a working WhatsApp task manager in about thirty minutes.

macOS (Apple Silicon & Intel) · Windows · Linux · Version 1.7

Illustrations are drawn representations of the real screens; exact appearance varies slightly by version.

1 Before you begin — what you need

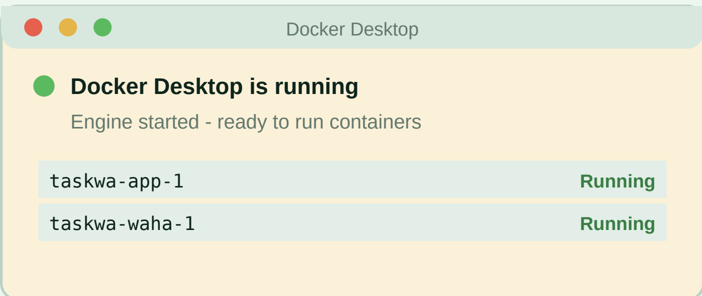
Item	Details
A computer that stays on	Mac (Apple Silicon or Intel), Windows 10/11, or Linux. 4 GB RAM is plenty. This machine is the server: reminders only go out while it is awake and Docker is running.
A WhatsApp number for the bot	Either a dedicated SIM (safest) or your own personal number (supported via Personal-number mode — understand the ban-risk trade-off in the User Manual first).
A phone with that number's WhatsApp	Needed once, to scan a QR code.
30 minutes	Most of it is Docker downloading things.

> Every command in this guide is also in INSTALL.txt in the project folder — open it side-by-side and copy-paste each block in order.

2 Install Docker Desktop

Docker runs TaskWA's three components in isolated containers — it is the only software you install yourself.

macOS: download from `docker.com/products/docker-desktop` — choose **Apple Silicon** for M1/M2/M3/M4 Macs, **Intel** otherwise. Open the .dmg, drag Docker into Applications, then launch it once and approve the system prompts. **Windows:** the installer enables WSL2; if it complains about virtualization, enable it in BIOS/UEFI (usually 'SVM' or 'VT-x'). **Linux:** `curl -fsSL https://get.docker.com | sh`



Checklist after first launch:

✓ Whale icon steady in the menu bar / tray
Settings > General > 'Start Docker Desktop when you sign in' ON
✓ Terminal: docker --version prints a version

Fig. 1 — Docker Desktop running, with both TaskWA containers up (as it will look after step 4).

3 Get TaskWA and set your secrets

Open Terminal (macOS: Cmd-Space, type Terminal · Windows: PowerShell) and run:

```
git clone https://github.com/sudhakar-kadavasal/taskwa.git   # or unzip the release
cd taskwa
```

Then paste this whole block — it creates your `.env`, generates all three secrets automatically, and detects your CPU to enable the ARM gateway image when needed (Apple Silicon Macs, Raspberry Pi). On Windows, run it in **Git Bash** (installed with Git), not PowerShell:

```
cp .env.example .env

for key in WEBHOOK_SECRET APP_SECRET WAHA_API_KEY; do
  val=$(openssl rand -hex 24)
  if [ "$(uname)" = "Darwin" ]; then sed -i '' "s|^$key=.*|$key=$val|" .env
  else sed -i "s|^$key=.*|$key=$val|" .env; fi
done

case "$(uname -m)" in arm64|aarch64)
  if [ "$(uname)" = "Darwin" ]; then sed -i '' 's|^# *WAHA_TAG=arm|WAHA_TAG=arm|' .env
  else sed -i 's|^# *WAHA_TAG=arm|WAHA_TAG=arm|' .env; fi ;;
esac

grep -q change-me .env && echo "!! secrets NOT set" || echo "secrets OK"
```

> The last line must print secrets OK. If it doesn't (or you can't use Git Bash), edit `.env` by hand: run `openssl rand -hex 24` three times, paste one result into each secret, and on Apple Silicon / Raspberry Pi uncomment `WAHA_TAG=arm` — without it Docker reports 'no matching manifest for linux/arm64'.

Optional — choose your WhatsApp engine. The default (WEBJS) is the right choice for almost everyone and needs no changes. If you already know you want a lighter engine (NOWEB or GOWS), `.env.example` has a fully commented picker — one block per engine and CPU architecture, each with a short why/why-not note. Uncomment exactly one block; leave the rest commented. Pick this now if you're going to — switching later means rebuilding the gateway container and scanning a fresh QR (see the Troubleshooting section).

WAHA_ENGINE	WAHA_TAG (amd64)	WAHA_TAG (arm64)
WEBJS (default)	(leave blank)	arm
NOWEB	noweb	noweb-arm
GOWS	gows	gows-arm

4 Start it

This step needs Docker **running** — you installed it in step 2, right? If not, get it now from `docker.com/products/docker-desktop`, launch it once, wait for the whale icon, then come back.

```
docker compose up -d
```

Terminal

```
$ docker compose up -d
[+] Running 2/2
 ✔ Container taskwa-waha-1   Started
 ✔ Container taskwa-app-1    Started
$ docker compose ps
NAME            STATUS
taskwa-app-1    Up 15 seconds
```

Fig. 2 — First start. The initial run downloads ~1 GB; give it a few minutes. Both containers must show 'Up'.

5 The setup wizard

Open **`http://localhost:3000`** in your browser. TaskWA starts in **dry-run mode** — nothing is sent to anyone until you switch it off, so you cannot accidentally spam your team while setting up.

Fig. 3 — The five wizard steps, in order.

- **Step 1 — Admin password.** Eight characters or more. This protects the dashboard.
- **Step 2 — Pair WhatsApp** (next section — the only step involving a phone).
- **Step 3 — Timezone and send time.** IANA name (Asia/Dubai, Asia/Kolkata, Europe/London) and the daily digest time, 24-hour clock. Several times? Comma-separate: 08:00, 17:30.
- **Step 4 — Team members.** Name + WhatsApp number, digits only with country code (9715xxxxxxxx). Add yourself first, as admin.
- **Step 5 — Test.** Send the test message; in dry-run it appears in the Health page log rather than on your phone — that is correct. Click Finish.
- **Using your own personal number as the bot?** After the wizard, open Settings and tick **Personal number mode** — your digest then arrives in WhatsApp's 'Message Yourself' chat, and the bot stays silent on everything that isn't a command.

6 Pairing WhatsApp — the QR scan

Fig. 4 — Browser shows the QR; the bot number's phone scans it.

If the QR does not appear: wait 60 seconds (the gateway boots a full browser engine internally — slower on Apple Silicon's first run), click *Start / restart session*, refresh. The status label above the QR tells you the state; you want it to end at **WORKING**.

7 Groups, members, first test, go live

- **Register groups.** Add the bot's number to each WhatsApp group you want covered. Dashboard → Groups → **Detect my groups** → Register next to each. Unregistered groups are ignored entirely.
- **Import members from a group.** Members → **Import from group** → pick a registered group: every participant appears with name and number pre-filled. Tick, adjust names, Import. (Participants whose numbers WhatsApp hides need to be saved as a contact, or make the bot's number a group admin, then re-detect.)
- **Personal number?** Settings → tick **Personal number mode** → Save. Your own digest will arrive in WhatsApp's 'Message Yourself' chat.
- **Dry-run rehearsal.** Health → **Run digests now** → read the message log. What you see there is exactly what would have been sent.
- **Go live.** Settings → untick Dry-run → Save. Health → Run digests now — this time the messages land on phones.
- **Prove the loop.** From your phone: /mytasks → create a test task with /add → reply 1 done → watch for the thumbs-up and the dashboard updating.
- **See the new v1.7 board.** Send **/board** from your phone for a picture of your own tasks, or open Dashboard → Board for the kanban view and the Preview panel (sends board images to you only, never the team — see the User Manual, Part II).

8 Keep it alive — fully unattended

Reminders only go out while the machine is on, Docker is running, and the containers are up. After a reboot or power cut that chain has four links; the scripts below repair the one Docker's own restart policies miss (containers that were stopped when Docker last quit). One command, from the install folder:

```
# macOS - LaunchAgent at every login (also starts Docker itself):
./scripts/autostart-macos.sh

# Windows - Scheduled Task at logon (PowerShell):
powershell -ExecutionPolicy Bypass -File scripts\autostart-windows.ps1

# Linux / Raspberry Pi - systemd service, no login needed at all:
sudo ./scripts/autostart-linux.sh
```

Each script waits up to five minutes for the Docker engine, then runs `docker compose up -d` (idempotent — if everything is already running it does nothing). Add `--uninstall` (or `-Uninstall`) to remove.

Complete the chain once: **auto power-on** after a power cut (macOS: `sudo pmset -a autorestart 1`; PCs: 'Restore on AC power' in BIOS), and **automatic login** (macOS: System Settings → Users & Groups; Windows: netplwiz). With FileVault/BitLocker a password is still needed at cold boot — accept that; don't disable disk encryption. Verify the whole chain: reboot, touch nothing, wait three minutes, open the Health page — it should say WORKING. Details: **docs/UNATTENDED.md**.

> What the app itself already handles: task digests missed by under six hours are sent on startup; missed broadcasts are deliberately skipped (a scheduled 'good morning' should not arrive mid-afternoon); the WhatsApp pairing survives restarts.

9 If something goes wrong

Symptom	Fix
docker: command not found	Docker Desktop isn't installed or was never launched. Launch it, wait for the whale, open a NEW terminal window.
no matching manifest for linux/arm64	Apple Silicon / Raspberry Pi: add WAHA_TAG=arm to .env, then docker compose up -d
QR never appears on first start	Wait 60 s (the gateway boots a full browser), then Start session, refresh. Check: docker compose logs waha --tail 50.
Session goes FAILED or STOPPED	Health page → Restart gateway session — restarts the session, pairing kept, no QR. Recovery is manual (a click), never automatic. Still failing: docker compose restart waha.
A QR appears after a plain restart	Not a bug — WhatsApp logged the number out. Health page → Re-pair (new QR), scan. Recurring every few days? Keep the phone online and don't open the bot's number in WhatsApp Web/Desktop elsewhere.
Banner: session UNREACHABLE right after start	Normal for the first minute while the gateway boots. Refresh the Health page.
Replies do nothing	Is the sender registered with the exact number they message from? Is the group registered? Check docker compose logs app — every message and every drop reason is logged.
Locked out of dashboard	'Forgot password' sends a code to the admin's WhatsApp. Gateway down too: docker compose exec app python -m app.cli reset-password
Want to switch WhatsApp engine (WEBJS/NOWEB/GOWS)	Not a dashboard toggle — edit the picker in .env, then docker compose up -d --force-recreate waha, then re-pair (new QR expected). Full procedure in docs/TROUBLESHOOTING.md.

The restart cookbook — from the install folder; none of these lose data (database, backups and the WhatsApp pairing live in `data/` and survive):

```
# First try the Health page 'Restart gateway session' button (restarts
# the WhatsApp session, no QR). These are the heavier hammers if that
# doesn't recover it:
docker compose restart waha        # gateway still stuck after the button
docker compose restart app         # dashboard misbehaving
docker compose down && docker compose up -d    # full clean restart
docker compose logs app --since 30m            # what actually happened
docker stats waha --no-stream                  # RAM: is Chromium being OOM-killed?

# Docker itself wedged: quit Docker Desktop (whale menu), reopen,
# wait for the steady whale, then:  docker compose up -d
```

Check the autostart is armed (it is what brings everything back after reboots and power cuts): macOS `launchctl list | grep taskwa` should print a line — if not, re-run `./scripts/autostart-macos.sh` (log: `/tmp/taskwa-autostart.log`). Also confirm Docker Desktop's 'Start when you sign in' is ON, automatic login is enabled, and on a Mac run `sudo pmset -a autorestart 1` once. 'docker: command not found' after switching container engines: Docker Desktop → Settings → Advanced → toggle CLI tools User → System (password prompt). Never run two container engines at once.

Full operational reference: **docs/TROUBLESHOOTING.md**, **docs/UPGRADE.md**, **docs/BACKUP.md**, **docs/UNATTENDED.md** and the User Manual in this folder.